

Lecture Unit 10.4

GPU programming for HPC

Programming a GPU for HPC

- Programming a GPU requires some different considerations compared to programming for a CPU
 - Data movement
 - Hiding memory latency
 - Avoiding branching
- Programming models

Data movement

- The CPU and GPU address separate memory
- GPU memory smaller than host memory
- Need to transfer data between CPU memory and GPU memory
- Transfer is over the PCIe bus is slow: need to minimise transfers
- Small tasks may be faster on CPU: there is a minimum granularity for offloading to be beneficial
- Need to consider data locality to get good performance

Hiding memory latency

- CPUs hide memory latency using cache
- GPUs over-commit cores to hide memory latency
- Context switching is much faster on a GPU than a CPU (over-committing cores is not beneficial to performance on a CPU)
- Require a higher degree of parallelism for GPUs

Avoiding branching

- GPUs are suited to data parallel workloads where each thread carries out the same operation on different data items (Single Instruction Multiple Data)
- Branching is bad for performance
 - Branch divergence within a warp causes performance degradation as threads need to execute in lock step
 - GPUs do not have the branch prediction available in CPUs

Programming models

- Traditional programming languages alone are not able to make use of GPUs so other programming models have been developed:
 - [OpenMP](#): compiler directives (C/C++, Fortran)
 - [OpenCL](#): C based
 - [OpenACC](#): compiler directives (C/C++, Fortran)
 - [CUDA](#): C/C++, Fortran based (from NVIDIA)
 - [HIP](#): C++ based (from AMD)
 - [Data Parallel C++](#) (from Intel)

OpenMP

- OpenMP 4.0 and later include offloading to accelerators via the **target** construct:
- The **teams distribute** directive distributes loop iterations over threads in a team
- Combined construct does both:

```
#pragma omp target teams distribute
```

Example

Vector addition:

```
#include <stdio.h>
#define N (2048*2048)

int main()
{
    int a[N], b[N], c[N];

    for( int i=0; i < N; i++ ){
        a[i] = i;
        b[i] = i;
    }

    #pragma omp target teams distribute
    for( int i = 0; i < N; i++ )
    {
        c[i] = a[i] + b[i];
    }

    printf( "c[%d] = %d\n",0,c[0] );
    printf( "c[%d] = %d\n",N-1, c[N-1] );

    return 0;
}
```

← Offload to GPU and distribute
work over threads in a team

OpenMP

Run under the `nvprof` profiler

```
> nvprof ./vadd_omp
==17958== NVPROF is profiling process 17958, command: ./vadd_omp
c[0] = 0
c[4194303] = 8388606
==17958== Profiling application: ./vadd_omp
==17958== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	49.83%	5.7444ms	1	5.7444ms	5.7444ms	5.7444ms	[CUDA memcpy DtoH]
	49.36%	5.6901ms	3	1.8967ms	1.7549ms	1.9890ms	[CUDA memcpy HtoD]
	0.81%	93.855us	1	93.855us	93.855us	93.855us	main\$ck_L14_1
API calls:	93.89%	231.76ms	1	231.76ms	231.76ms	231.76ms	cuCtxCreate
	2.91%	7.1899ms	1	7.1899ms	7.1899ms	7.1899ms	cuMemcpyDtoH
	2.38%	5.8638ms	3	1.9546ms	1.8047ms	2.0700ms	cuMemcpyHtoD
	0.47%	1.1673ms	1	1.1673ms	1.1673ms	1.1673ms	cuMemHostAlloc
	0.23%	558.84us	3	186.28us	178.32us	192.02us	cuMemAlloc
	0.05%	113.62us	1	113.62us	113.62us	113.62us	cuModuleLoadData
	0.03%	84.540us	1	84.540us	84.540us	84.540us	cuStreamSynchronize
	0.02%	44.729us	1	44.729us	44.729us	44.729us	cuLaunchKernel
	0.01%	17.012us	34	500ns	294ns	3.5320us	cuEventCreate
	0.01%	13.017us	1	13.017us	13.017us	13.017us	cuStreamCreate
	0.01%	12.376us	2	6.1880us	4.4530us	7.9230us	cuDeviceGetPCIBusId
	0.00%	4.3650us	10	436ns	329ns	787ns	cuDeviceGetAttribute
	0.00%	3.0250us	3	1.0080us	307ns	1.7980us	cuDeviceGetCount
	0.00%	2.8900us	1	2.8900us	2.8900us	2.8900us	cuMemHostGetDevicePointer
	0.00%	2.6620us	3	887ns	381ns	1.2080us	cuDeviceGet
	0.00%	1.9800us	1	1.9800us	1.9800us	1.9800us	cuModuleGetGlobal
	0.00%	1.9540us	1	1.9540us	1.9540us	1.9540us	cuModuleGetFunction
	0.00%	985ns	1	985ns	985ns	985ns	cuCtxGetCurrent
	0.00%	946ns	2	473ns	159ns	787ns	cuFuncGetAttribute
	0.00%	797ns	1	797ns	797ns	797ns	cuCtxSetCurrent
	0.00%	691ns	1	691ns	691ns	691ns	cuFuncSetCacheConfig

> █

1x device to host copy

3x host to device copy

Example

Vector addition with map clause:

```
#include <stdio.h>
#define N (2048*2048)

int main()
{
    int a[N], b[N], c[N];

    for( int i=0; i < N; i++ ){
        a[i] = i;
        b[i] = i;
    }

    #pragma omp target data map(to:a,b) map(from:c) ← Specify data transfer
    #pragma omp target teams distribute               to and from device
    for( int i = 0; i < N; i++ )                     explicitly
    {
        c[i] = a[i] + b[i];
    }

    printf( "c[%d] = %d\n",0,c[0] );
    printf( "c[%d] = %d\n",N-1, c[N-1] );

    return 0;
}
```

OpenMP

```
> nvprof ./vadd_openmp_withMap
==18230== NVPROF is profiling process 18230, command: ./vadd_openmp_withMap
c[0] = 0
c[4194303] = 8388606
==18230== Profiling application: ./vadd_openmp_withMap
==18230== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	58.76%	5.7009ms	1	5.7009ms	5.7009ms	5.7009ms	[CUDA memcpy DtoH]
	40.26%	3.9061ms	2	1.9531ms	1.9380ms	1.9681ms	[CUDA memcpy HtoD]
	0.98%	94.591us	1	94.591us	94.591us	94.591us	main\$ck_L15_1
API calls:	94.71%	233.41ms	1	233.41ms	233.41ms	233.41ms	cuCtxCreate
	2.84%	6.9885ms	1	6.9885ms	6.9885ms	6.9885ms	cuMemcpyDtoH
	1.63%	4.0193ms	2	2.0096ms	1.9823ms	2.0370ms	cuMemcpyHtoD
	0.47%	1.1704ms	1	1.1704ms	1.1704ms	1.1704ms	cuMemHostAlloc
	0.22%	552.12us	3	184.04us	177.39us	189.05us	cuMemAlloc
	0.05%	111.35us	1	111.35us	111.35us	111.35us	cuModuleLoadData
	0.03%	84.766us	1	84.766us	84.766us	84.766us	cuStreamSynchronize
	0.02%	45.030us	1	45.030us	45.030us	45.030us	cuLaunchKernel
	0.01%	17.034us	34	501ns	287ns	3.3490us	cuEventCreate
	0.01%	13.123us	1	13.123us	13.123us	13.123us	cuStreamCreate
	0.01%	12.380us	2	6.1900us	4.3040us	8.0760us	cuDeviceGetPCIBusId
	0.00%	4.3070us	10	430ns	321ns	816ns	cuDeviceGetAttribute
	0.00%	3.2210us	1	3.2210us	3.2210us	3.2210us	cuMemHostGetDevicePointer
	0.00%	3.0220us	3	1.0070us	298ns	1.7070us	cuDeviceGetCount
	0.00%	2.5930us	3	864ns	433ns	1.1000us	cuDeviceGet
	0.00%	2.0730us	1	2.0730us	2.0730us	2.0730us	cuModuleGetFunction
	0.00%	1.9400us	1	1.9400us	1.9400us	1.9400us	cuModuleGetGlobal
	0.00%	960ns	2	480ns	158ns	802ns	cuFuncGetAttribute
	0.00%	939ns	1	939ns	939ns	939ns	cuCtxGetCurrent
	0.00%	801ns	1	801ns	801ns	801ns	cuCtxSetCurrent
	0.00%	670ns	1	670ns	670ns	670ns	cuFuncSetCacheConfig

> █

1x device to host copy (points to [CUDA memcpy DtoH])

2x host to device copy (points to [CUDA memcpy HtoD])

Unit summary

- GPU programming has some different considerations compared to CPU programming: data movement, avoiding branching
- GPU programming requires additions to standard programming languages
- A number of different programming models are available some of which are vendor specific
- OpenMP 4.0 added offloading and provides a multi-vendor method for using GPUs
- The **map** clause can be used to controlling data movement to improve performance